Scoring Functions

Validating Twin Drill Holes for Resource Estimation

Matthew Nimmo

2026-06-17

Abstract Geological data validation can include the use of twin drill holes to verify historic drill hole data, verify high-grade assay intersections, and check for sampling bias. Traditionally, assessing these paired data involves subjective visual inspections of scatter plots, quantile-quantile plots, and down-hole logs or the use of global statistics–a process that does not scale across large databases. This notebook introduces the foundational concept of Scoring Functions as a mechanism to quantify data integrity. To overcome the problem of scaling the analysis, Scoring Functions are introduced to illustrate how the observed geological variability between the paired data can be translated to a normalized, deterministic metric, which can then be used as a standardized quality KPI. A simple synthetic dataset that contains two drill holes, treated as a twin pair, with lithology and assays for copper, iron, and sulphur is used to demonstrate the use of the Scoring Function. The example and notebook forms a reproducible R framework for modular data assessment, bridging the gap between traditional geological quality control and scalable data engineering.

Table of contents

What is a Scoring Function?

A scoring function is a mathematical formula or algorithm that takes a set of data inputs and maps them to a single numerical value (a "score"). This score represents the quality, probability, value, or rank of that specific input relative to others. In technical terms, the scoring function takes an input vector (a list of characteristics or features) and computes a scalar value (the score).

$$f(x_1, x_2, x_3, ..., x_n) = Score$$

Where: - $x_1, x_2, x_3, ..., x_n$ are the features - f is the scoring function

Scoring functions are used in:

1. Search engines (Information Retrieval)
2. Machine Learning and AI
3. Credit scoring (Finance)
4. Video game AI and chess bots
5. Data quality assessment

In data science, a scoring functions is often confused with a loss (or cost) function. They are closely related but move in opposite directions. With a scoring function you want to maximize the value. A higher score means a better, more accurate, or more desirable outcome. With a loss function you want to minimize the value. It measures the error of a system. A higher loss means the system is performing poorly.

Without scoring functions, automation would be incredibly difficult. They allow computers to quickly evaluate millions of possibilities, rank them from best to worst, and make data-driven choices in milliseconds. It allows us to quickly explore and assess data quality without the need to manually inspect the data visually. It also allows us to then semi-automate the processing of messy data by endeavouring to improve the score through data wrangling.

In the context of Data Quality Assessment (DQA), a scoring function acts as a health check for the data. It ingests complex, raw data, calculates metrics for observable data features such as missing values, outliers, duplicate entries, data inconsistencies, and stale timestamps and synthesizes them into a single, standardized KPI (usually expressed as a percentage from 0% to 100%).

The score should be simple to construct and easy to understand. It should be possible to compare scores generated on different data sets and determine which data is better or where there are data issues. The score should not depend on size or shape of the data and be normalized with a clear scale between highest and lowest scores relative to a standard or expectation. And the score should be comparable across data sets–it can be used to benchmark data from a quality perspective.

A scoring function can consist of one or more rules or constraints that are applied to the data. A constraint could be an indicator or flag set to indicate values that must be positive or below a threshold value (minima) or above a threshold value (maxima) or contain no missing values. Or it could be a list of valid factors or acceptable reference values. Or it could be a correlation between multiple columns or restriction on the nature of the distribution (skew or kurtosis). Or it could be a restriction on duplicate values or rows. If any one of the constraints are violated then the resulting score should be closer to zero or be zero. For any rule violation (a problem) we can determine its prevalence–the relative frequency of the rule violation across a data set. Associated with a rules prevalence is its confidence. For explicit rules, rules that we know and have confirmed the confidence is 100%. For implicit rules, rules that are assumed by the system, are calculated from the rules prevalence and will be less than or equal to 100%.

There is no right or wrong way of defining a scoring function. The only real constraint on how the scoring function is defined is that when it has been defined it is immutable and applied consistently across all the data. This is to ensure that the score is comparable. An important feature if we want to use the score for benchmarking datasets or quantifying and assessing data quality.

In Data Quality Assessment the scoring function typically follows a modular, bottom-up calculation hierarchy. It is calculated systematically by rolling up metrics through three layers:

1. The Rule Level (Base Layer)
2. The Dimension Level (Middle Layer)
3. The Global Dataset Score (Top Layer)

How and why we use them

Instead of relying on subjective manual reviews, scoring functions allow us to quantify data quality. By implementing them, we can:

- Scale inspections by automatically evaluating thousands of data points.
- Establish baselines to track whether data quality is improving or degrading over time.

- Standardize Workflows to bridge the gap between subjective assessment and rigid data engineering.

  Effective scoring functions are deterministic. They rely on simple, repeatable rules. If you pass the same data through the function multiple times, you will always get the exact same score.

Case study: validating twin drill holes

Twin drilling is just one of many ways to validate geological data for use in Mineral Resource estimation. They are used to help test the repeatability and very short range variability of the geology and geochemistry and to check historic data. The comparison is usually done manually by plotting the paired data as log traces or scatter plots or quantile-quantile plots. Assessment of the results is usually subjective. But it doesn't have to be.

A basic twin drill hole comparison will be used to illustrate how scoring functions could be used to quantify the difference between the original drill hole and its twin.

1. Generating synthetic twin holes

We will generate a primary hole (`DH001`) and its twin (`DH001_TWIN`). They share identical lithology but have simulated variance in their copper ($Cu\%$), iron ($Fe\%$), and sulphur ($S\%$) assays.

```
library(tidyverse)
```

```
── Attaching core tidyverse packages ──────────────────────── tidyverse
2.0.0 ──
✓ dplyr     1.2.1     ✓ readr     2.2.0
✓ forcats   1.0.1     ✓ stringr   1.6.0
✓ ggplot2   4.0.3     ✓ tibble    3.3.1
✓ lubridate 1.9.5     ✓ tidyr     1.3.2
✓ purrr     1.2.2
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

```
library(ggplot2)
library(patchwork)

set.seed(11) # For repeatability

# Define intervals
depth_from <- seq(0, 100, by=10)
depth_to   <- seq(10, 110, by=10)
lithology  <- c("AAC", "AAC", "CC", "XNL", "XNL", "XNL", "XNL",
                "XRT", "SAM", "SAM", "SAM")

# True underlying grade trend
true_cu <- c(0.1, 0.2, 0.8, 1.2, 1.5, 1.1, 0.97, 0.4, 0.2, 0.1, 0.05)
true_fe <- c(6.5, 3.4, 17.5, 21.7, 23.5, 20.9, 18.9, 13.9, 1.3, 2.2, 1.8)
true_s <- c(1.3, 0.8, 12.3, 11.8, 17.1, 16.4, 17.2, 12.5, 0.8, 1.1, 0.5)

# Generate Primary Hole
dh_primary <- tibble(
  hole_id = "DH001",
  from = depth_from,
  to = depth_to,
  mid = (from + to) / 2,
  lith = lithology,
  cu_pct = pmax(0.01, true_cu + rnorm(11, mean=0, sd=0.08)),
  fe_pct = pmax(0.01, true_fe + rnorm(11, mean=0, sd=1.18)),
  s_pct = pmax(0.01, true_s + rnorm(11, mean=0, sd=3.22))
)

# Generate Twin Hole (with simulated assay/geological variance)
dh_twin <- tibble(
  hole_id = "DH001_TWIN",
  from = depth_from,
  to = depth_to,
  mid = (from + to) / 2,
  lith = lithology,
   cu_pct = pmax(0.01, true_cu + rnorm(11, mean=0, sd=0.24)), # Higher
variance
  fe_pct = pmax(0.01, true_fe + rnorm(11, mean=4.7, sd=1.18)), # Mean bias
   s_pct = pmax(0.01, true_s + rnorm(11, mean=-10.3, sd=6.22)) # Higher
variance -Mean bias
)
```

```
# Combine datasets
twin_pairs <- bind_rows(dh_primary, dh_twin)
twin_df <- dh_primary |>
      select(hole_id,   from,   to,   mid,   lith,   cu_primary=cu_pct,
fe_primary=fe_pct, s_primary=s_pct) |>
  inner_join(
         select(dh_twin,   from,   to,   cu_twin=cu_pct,   fe_twin=fe_pct,
s_twin=s_pct),
    by = c("from", "to")
  )
```

```
head(twin_pairs, 11)
```

```
# A tibble: 11 × 8
   hole_id  from    to   mid lith  cu_pct fe_pct s_pct
   <chr>   <dbl> <dbl> <dbl> <chr>  <dbl>  <dbl> <dbl>
 1 DH001       0    10     5 AAC   0.0527  6.09   0.01
 2 DH001      10    20    15 AAC   0.202   1.58   1.94
 3 DH001      20    30    25 CC    0.679  17.2   12.5
 4 DH001      30    40    35 XNL   1.09   20.3   11.8
 5 DH001      40    50    45 XNL   1.59   23.5   16.5
 6 DH001      50    60    55 XNL   1.03   20.6   13.9
 7 DH001      60    70    65 XNL   1.08   19.9   16.5
 8 DH001      70    80    75 XRT   0.450  13.2    9.33
 9 DH001      80    90    85 SAM   0.196   0.526  0.01
10 DH001      90   100    95 SAM   0.0197  1.39   0.01
11 DH001     100   110   105 SAM   0.01    1.78   2.69
```

2. Traditional visual comparison

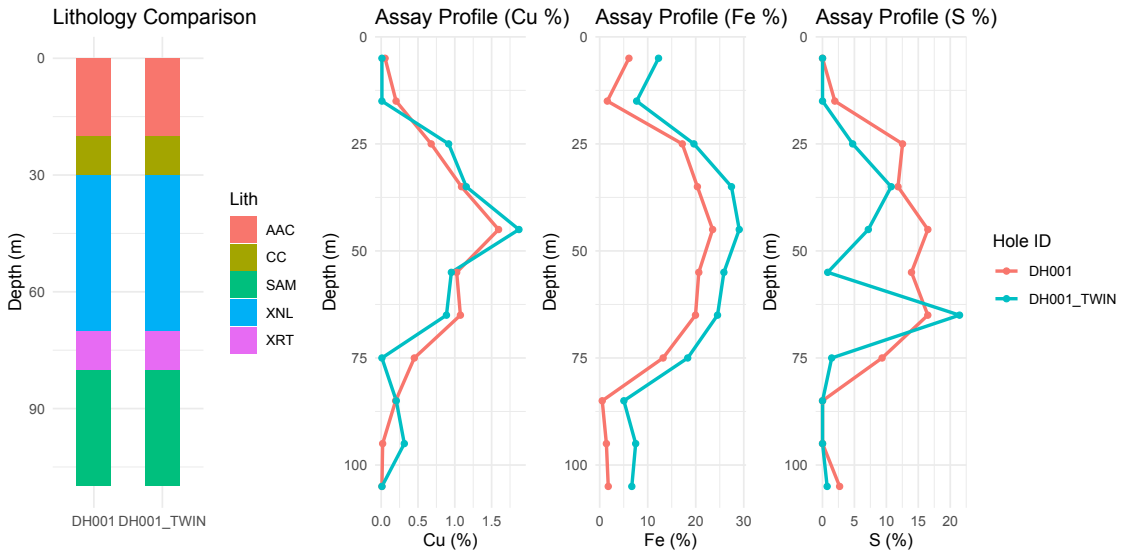
Before scoring, let's look at the data using the traditional approach of plotting the drill hole logs and geochemistry traces side-by-side and plotting the paired data points as scatter plots.

```
p1 <- ggplot(twin_pairs, aes(x=hole_id, y=mid, fill=lith)) +
  geom_tile(width = 0.5, stat = "identity") +
  scale_y_reverse() +
  labs(title="Lithology Comparison", x="", y="Depth (m)", fill="Lith") +
  theme_minimal()

p2 <- ggplot(twin_pairs, aes(x=mid, y=cu_pct, color=hole_id)) +
```

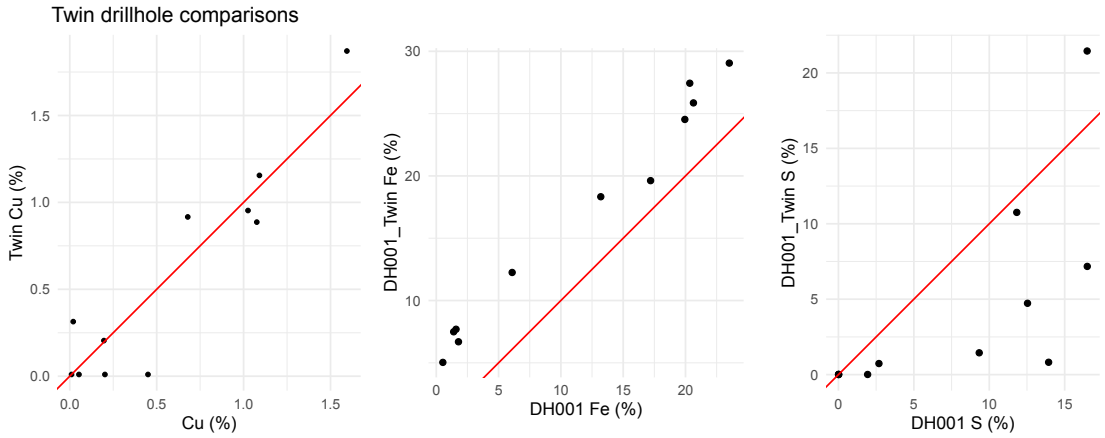
```
  geom_line(size = 1) +
  geom_point() +
  coord_flip() +
  scale_x_reverse() +
 labs(title="Assay Profile (Cu %)", x="Depth (m)", y="Cu (%)", color="Hole
ID") +
  theme_minimal() +
  theme(legend.position="none")
```

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
ℹ Please use `linewidth` instead.
```

```
p3 <- ggplot(twin_pairs, aes(x=mid, y=fe_pct, color=hole_id)) +
  geom_line(size = 1) +
  geom_point() +
  coord_flip() +
  scale_x_reverse() +
 labs(title="Assay Profile (Fe %)", x="Depth (m)", y="Fe (%)", color="Hole
ID") +
  theme_minimal() +
  theme(legend.position="none")

p4 <- ggplot(twin_pairs, aes(x=mid, y=s_pct, color=hole_id)) +
  geom_line(size = 1) +
  geom_point() +
  coord_flip() +
  scale_x_reverse() +
  labs(title="Assay Profile (S %)", x="Depth (m)", y="S (%)", color="Hole
ID") +
  theme_minimal()

p1 + p2 + p3 + p4 + plot_layout(ncol=4)
```




```
p1 <- ggplot(twin_df, aes(x=cu_primary, y=cu_twin)) +
  geom_point(size = 1) +
  geom_abline(intercept=0, slope=1, color="red") +
  coord_fixed(ratio=1) +
  labs(title="Twin drillhole comparisons", x="Cu (%)", y="Twin Cu (%)") +
  theme_minimal() +
  theme()

p2 <- ggplot(twin_df, aes(x=fe_primary, y=fe_twin)) +
  geom_point() +
  geom_abline(intercept=0, slope=1, color="red") +
  coord_fixed(ratio=1) +
  labs(x="DH001 Fe (%)", y="DH001_Twin Fe (%)") +
  theme_minimal() +
  theme()

p3 <- ggplot(twin_df, aes(x=s_primary, y=s_twin)) +
  geom_point() +
  geom_abline(intercept=0, slope=1, color="red") +
  coord_fixed(ratio=1) +
  labs(x="DH001 S (%)", y="DH001_Twin S (%)") +
  theme_minimal()

p1 + p2 + p3 + plot_layout(ncol=3, widths=c(1,1,1))
```



3. Applying the scoring function

While visuals are great for one pair, they don't scale to 50 twin pairs.

Let's build a deterministic scoring function.

Rule:

If the relative difference between the original drill hole assay and its twin is less than or equal to a threshold value the indicator value is (0). Otherwise, it is (1). An indicator value of (1) is a rule violation and suggests that there is likely a problem with the cell value.

The choice of how we calculate the relative difference matters. For very small numbers close to assay detection limits, example 0.01 and 0.03, the relative difference is very high which would cause the rule to indicate a failure (0). While for large values the rule may return a pass even though the difference is significant. Using absolute standard relative difference or log relative difference does not change this. To overcome the problem without making the rule mathematically complicated we can use the Damped Relative Difference or Max-Normalized Bounded Score.

$$\text{Damped Relative Difference} = \frac{\mid A - B \mid}{\frac{(A+B)}{2} + c}$$

$$\text{Max-Normalized Bounded Score} = \frac{\mid A - B \mid}{max(A, B) + c}$$

Where: - A is the original value - B is the twin value - c is a small constant

The choice of threshold value also matters. A small threshold value would result in low scores while a large threshold could result in a perfect score every time.

Both cases are not appropriate for flagging data quality problems that need to be handled. We could consider the Thompson-Howarth error analysis and use a proportional threshold but we still need to specify the percentage acceptable error.

$$Threshold(x) = d + (p * x)$$

Where: - `d` is the absolute detection limit - `p` is the percentage acceptable error - `x` is the average of the two numbers $(A + B)/2$

Score:

The score for an individual cell is normalized between 0 and 1, where 1 indicates perfect quality and 0 indicates maximum failure. If an interval passes the rule (indicator is 0), its cell score is 1. If an interval violates the rule (indicator is 1), it receives a penalty based on the rule's prevalence (the failure rate of that variable across the dataset). The penalized score is calculated as $1 - prevalence$. The final score for a specific element (column, row, or the whole project) is the average of these cell scores, easily translatable into a 0% to 100% KPI.

```
n <- nrow(twin_df)

# Define the scoring function parameters
THRESHOLD <- 0.5 # 50% relative difference
DAMPING <- 0.1

# We could calculate the DAMPING value from the data using lower 10th
percentile
c_cu <- quantile(twin_df$cu_primary, probs=0.1)
c_fe <- quantile(twin_df$fe_primary, probs=0.1)
c_s <- quantile(twin_df$s_primary, probs=0.1)

# Calculate indicator = there is a problem
rule_df <- twin_df |>
  mutate(
                        cu_diff      =      abs(cu_primary-cu_twin)      /
(pmax(cu_primary,cu_twin)+DAMPING),
                        fe_diff      =      abs(fe_primary-fe_twin)      /
(pmax(fe_primary,fe_twin)+DAMPING),
    s_diff = abs(s_primary-s_twin) / (pmax(s_primary,s_twin)+DAMPING),
    # The scoring function logic (Indicator Variable)
    cu_indicator = if_else(cu_diff <= THRESHOLD, 0, 1),
    fe_indicator = if_else(fe_diff <= THRESHOLD, 0, 1),
    s_indicator = if_else(s_diff <= THRESHOLD, 0, 1)
```

```
  )

# Calculate prevalence
# Confidence = (1 - prevalence)
prev_df <- rule_df |>
  summarise(
    cu_prev = sum(cu_indicator) / n,
    fe_prev = sum(fe_indicator) / n,
    s_prev = sum(s_indicator) / n
  )

# Calculate cell score
scored_df <- rule_df |>
  mutate(
     cu_score = if_else(cu_indicator == 1, 1 - prev_df$cu_prev, 1), #
Probability that there is no problem
    fe_score = if_else(fe_indicator == 1, 1 - prev_df$fe_prev, 1),
    s_score = if_else(s_indicator == 1, 1 - prev_df$s_prev, 1)
  ) |>
  mutate(
    score = (cu_score + fe_score + s_score) / 3
  )

# Calculate the final project/pair score
final_cu_score <- mean(scored_df$cu_score)
final_fe_score <- mean(scored_df$fe_score)
final_s_score <- mean(scored_df$s_score)
final_score <- (final_cu_score + final_fe_score + final_s_score) / 3

knitr::kable(scored_df[c("from","to","cu_score","fe_score","s_score","score")])
```

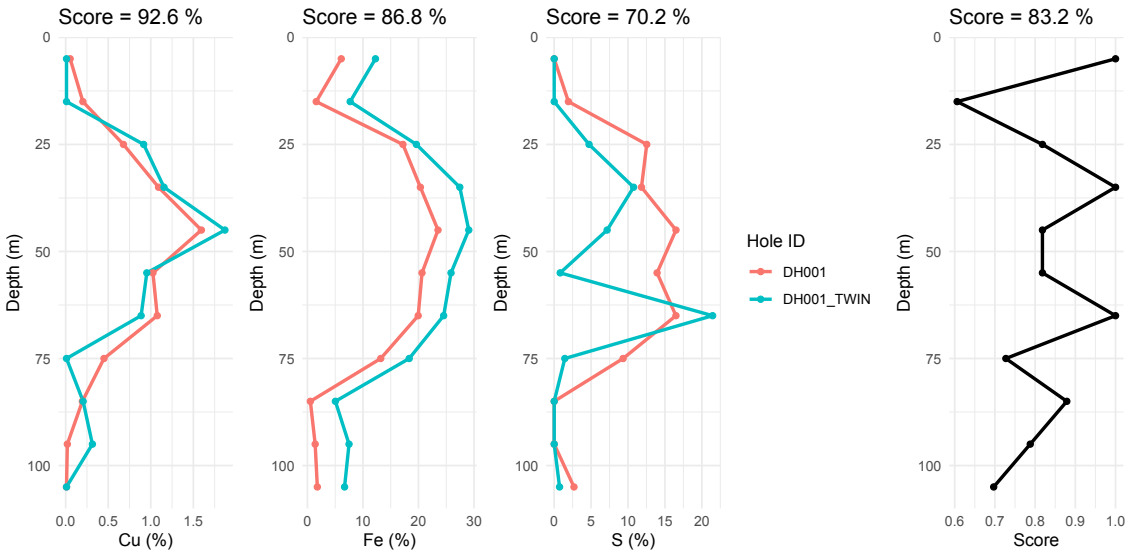
from	to	cu_score	fe_score	s_score	score
0	10	1.0000000	1.0000000	1.0000000	1.0000000
10	20	0.7272727	0.6363636	0.4545455	0.6060606
20	30	1.0000000	1.0000000	0.4545455	0.8181818
30	40	1.0000000	1.0000000	1.0000000	1.0000000
40	50	1.0000000	1.0000000	0.4545455	0.8181818
50	60	1.0000000	1.0000000	0.4545455	0.8181818
60	70	1.0000000	1.0000000	1.0000000	1.0000000

from	to	cu_score	fe_score	s_score	score
70	80	0.7272727	1.0000000	0.4545455	0.7272727
80	90	1.0000000	0.6363636	1.0000000	0.8787879
90	100	0.7272727	0.6363636	1.0000000	0.7878788
100	110	1.0000000	0.6363636	0.4545455	0.6969697

Final Twin Validation Score: 83% of intervals met the acceptable variance threshold.

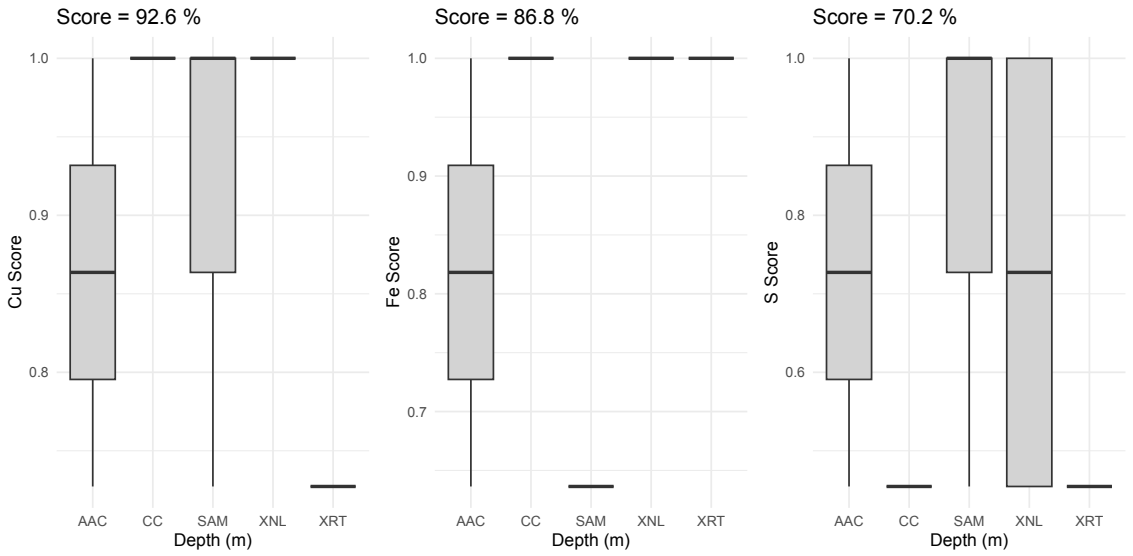
```
p1 <- ggplot(twin_pairs, aes(x=mid, y=cu_pct, color=hole_id)) +
  geom_line(size = 1) +
  geom_point() +
  coord_flip() +
  scale_x_reverse() +
 labs(title=paste("Score =", signif(final_cu_score,3)*100, "%"), x="Depth
(m)", y="Cu (%)", color="Hole ID") +
  theme_minimal() +
  theme(legend.position="none")

p2 <- ggplot(twin_pairs, aes(x=mid, y=fe_pct, color=hole_id)) +
  geom_line(size = 1) +
  geom_point() +
  coord_flip() +
  scale_x_reverse() +
 labs(title=paste("Score =", signif(final_fe_score,3)*100, "%"), x="Depth
(m)", y="Fe (%)", color="Hole ID") +
  theme_minimal() +
  theme(legend.position="none")

p3 <- ggplot(twin_pairs, aes(x=mid, y=s_pct, color=hole_id)) +
  geom_line(size = 1) +
  geom_point() +
  coord_flip() +
  scale_x_reverse() +
 labs(title=paste("Score =", signif(final_s_score,3)*100, "%"), x="Depth
(m)", y="S (%)", color="Hole ID") +
  theme_minimal()

p4 <- ggplot(scored_df, aes(x=mid, y=score)) +
  geom_line(size = 1) +
```

```
  geom_point() +
  coord_flip() +
  scale_x_reverse() +
   labs(title=paste("Score =", signif(final_score,3)*100, "%"), x="Depth
(m)", y="Score") +
  theme_minimal() +
  theme(legend.position="none")

p1 + p2 + p3 + p4 + plot_layout(ncol=4)
```




```
p1 <- ggplot(scored_df, aes(x=lith, y=cu_score)) +
  geom_boxplot(fill="lightgrey") +
 labs(title=paste("Score =", signif(final_cu_score,3)*100, "%"), x="Depth
(m)", y="Cu Score") +
  theme_minimal()

p2 <- ggplot(scored_df, aes(x=lith, y=fe_score)) +
  geom_boxplot(fill="lightgrey") +
 labs(title=paste("Score =", signif(final_fe_score,3)*100, "%"), x="Depth
(m)", y="Fe Score") +
  theme_minimal()

p3 <- ggplot(scored_df, aes(x=lith, y=s_score)) +
  geom_boxplot(fill="lightgrey") +
  labs(title=paste("Score =", signif(final_s_score,3)*100, "%"), x="Depth
```

```
(m)", y="S Score") +
  theme_minimal()

p1 + p2 + p3 + plot_layout(ncol=3)
```



Putting it all together

The scoring function. Whether you are interested in the cell scores, the column scores, or the final score the scoring function can be built to handle one or all of those. Here is the scoring function generalised to assessing twin drill hole columns. The function takes as arguments two vectors (the column assays) and outputs a single score in the range of 0 to 1. This function assumes that the two vectors are of the same length and both vectors are numeric. The function also assumes that the first vector `x` is the original drill hole and the second `y` is the twin.

```
score_twindh <- function(x, y) {
  if(!is.numeric(x) || !is.numeric(y)) {
    stop("data must be numeric")
  }
  if(length(x) != length(y)) {
    stop("data must be the same length")
  }

  # Define the scoring function parameters
  THRESHOLD <- 0.5 # 50% relative difference
```

```
  DAMPING <- 0.1

  n <- length(x)
  p10 <- quantile(x, probs=0.1) # Get the 10th percentile of the first
vector
  damp <- max(DAMPING, p10)

  # Calculate inverse score
  # Rule 1
  diff <- abs(x-y) / (pmax(x,y)+damp)
  ind <- ifelse(diff <= THRESHOLD, 0, 1) # Calculate indicator
  prev <- sum(ind) / n # Calculate prevalence
  iscore <- ind * prev # Calculate inverse score

  # Calculate other rules here
  # Replicate Rule 1 pattern
  # Combine cell rules by taking the product of all iscore values

  # Calculate score
  score <- 1 - iscore # Return this if interested in cell scores
  mean(score)
}
```

To use the function:

```
cu_score <- score_twindh(twin_df$cu_primary, twin_df$cu_twin)
fe_score <- score_twindh(twin_df$fe_primary, twin_df$fe_twin)
s_score <- score_twindh(twin_df$s_primary, twin_df$s_twin)

signif((cu_score + fe_score + s_score) / 3, 3) * 100
```

```
[1] 83.2
```

Going beyond a single drill hole

By keeping the scoring function simple, repeatable, and generic, we unlock powerful operational advantages: such as project-wide benchmarking, logging consistency, and even extending traditional QAQC.

You can calculate this score across every twin drill hole pair in the geological database, aggregating them to get a global “Confidence Score”. You can use the same framework to score logging code alignment between different geologists on

the same project to isolate logging bias. Instead of reviewing isolated scatter plots, you can aggregate scores by assay laboratory. If Lab A consistently scores 45/100 on twin pairs while Lab B scores 85/100, you have quantified a systematic calibration issue.